

Pokemon ETL — Pipeline de Datos con PokéAPI

Proyecto de portafolio que implementa un pipeline ETL completo consumiendo la PokéAPI, limpiando los datos y generando visualizaciones analíticas.

¿Qué hace este proyecto?

Fase	Descripción
Extract	Consume la PokéAPI (REST, sin auth) y descarga datos de
Transform	Normaliza estructuras JSON anidadas, elimina duplicados, imputa
Load	Exporta a CSVs y SQLite y genera reportes y visualizaciones con Plotly y Matplotlib

Estructura del Proyecto

```
Proyecto 2 CV/
├── README.md           ← Estás aquí
├── TASK.md             ← Guía de ejecución paso a paso
├── main.py             ← Entry point del pipeline completo
├── config.py           ← Configuración centralizada (URLs, límites, rutas)
├── requirements.txt    ← Dependencias del proyecto
├── .env.example        ← Plantilla de variables de entorno
├── .gitignore
├──
├── src/                ← Código fuente modular
│   ├── extract/
│   │   └── api_client.py ← Cliente HTTP para PokéAPI (con retry y rate limiting)
│   ├── transform/
│   │   └── cleaner.py    ← Normalización y limpieza de datos
│   └── load/
│       └── exporter.py   ← Exportación CSV, SQLite y reportes
├──
├── notebooks/         ← Análisis exploratorio y visualizaciones
│   ├── 01_exploration.ipynb ← Exploración inicial del dataset
│   ├── 02_analysis.ipynb   ← Análisis estadístico
│   └── 03_visualization.ipynb ← Dashboards y gráficas finales
├──
├── data/
│   ├── raw/           ← Datos crudos de la API (JSON)
│   ├── processed/     ← Datos limpios y transformados (CSV/Parquet)
│   └── output/        ← Reportes finales y exports
├──
├── tests/             ← Tests unitarios por módulo
│   ├── test_extract.py
│   ├── test_transform.py
│   └── test_load.py
├──
└── logs/              ← Logs de cada ejecución del pipeline
```

Fases del Pipeline ETL

Fase 1 — Extract

- Consume `/pokemon?limit=N` para listar Pokémon
- Descarga detalles individuales: stats base, tipos, habilidades, movimientos, altura/peso
- Rate limiting automático (respetar los límites de PokéAPI: ~100 req/min)
- Guarda JSON crudo en `data/raw/`

Fase 2 — Transform

- Desanida estructuras JSON complejas (stats, types, abilities son arrays)
- Normaliza nombres a `snake_case`
- Calcula campos derivados: BST (Base Stat Total), ratio peso/altura, categoría de poder
- Clasifica generaciones por rango de ID
- Exporta a `data/processed/pokemon_clean.csv`

Fase 3 — Load & Visualize

- Carga el CSV procesado en SQLite (`data/output/pokemon.db`) para queries ad-hoc
- Genera gráficas: distribución de tipos, correlación de stats, top Pokémon por BST
- Dashboard HTML interactivo con Plotly

Análisis que produce el pipeline

Análisis	Descripción
Distribución de tipos	¿Cuáles tipos son más comunes?
Top 20 por BST	Los Pokémon con mejores estadísticas totales
Correlación stats	¿Ataque vs Defensa, Velocidad vs BST?
Distribución por generación	¿Cuántos Pokémon por generación?
Peso vs Altura	Scatter con colores por tipo primario

Tecnologías

Librería	Uso
<code>`requests`</code>	Consumo de la API REST
<code>`pandas`</code>	Manipulación y limpieza de datos
<code>`plotly`</code>	Visualizaciones interactivas
<code>`matplotlib`</code> / <code>`seaborn`</code>	Gráficas estáticas
<code>`sqlite3`</code>	Almacenamiento relacional local
<code>`python-dotenv`</code>	Gestión de variables de entorno
<code>`tenacity`</code>	Retry automático en llamadas HTTP
<code>`loguru`</code>	Logging estructurado
<code>`pytest`</code>	Tests unitarios

Inicio Rápido

```
# 1. Crear entorno virtual
python -m venv venv && source venv/bin/activate

# 2. Instalar dependencias
pip install -r requirements.txt

# 3. Configurar entorno
cp .env.example .env

# 4. Ejecutar el pipeline completo
python main.py

# 5. Ejecutar solo una fase
python main.py --phase extract
python main.py --phase transform
python main.py --phase load

# 6. Explorar con notebooks
jupyter notebook notebooks/
```

PokéAPI: <https://pokeapi.co/api/v2/> — Pública, sin autenticación, límite 100 req/min.